PART 1— Docker Image

6. Application Build

Ippudu Kubernetes gurinchi kaadu. First mana application ni Docker image ga package cheyyali.
Java example:
mvn clean package
Build successful ayyaka:
target/myapp.jar
ane application JAR create avtundi.

---

7. Dockerfile Create

Application project lo Dockerfile create chestam.

```
FROM eclipse-temurin:17-jre
WORKDIR /app
COPY target/myapp.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Dockerfile anedi **image ela create cheyyalo instructions**. Idi container kaadu, image kaadu. Docker ee instructions ni use chesi image build chestundi.

---

8. Docker Image Build

```
docker build -t myapp:1.0 .
```

Ee command Dockerfile ni read chesi application JAR, Java runtime and required files ni package chesi:
myapp:1.0
ane Docker image create chestundi.
Check:

```
docker images
```

---

9. Local Container Test

Image correct ga work avtundo local ga test cheyyachu.

```
docker run -d \
  --name myapp-container \
  -p 8080:8080 \
  myapp:1.0
```

Ikkada **Docker image nunchi local container** create avtundi. Application container lopala run avtundi.
Check:

```
docker ps
```

Testing kosam idi. Production EKS lo manam later docker run use chesi containers create cheyyam.

PART 2 — ECR

10. ECR Repository Create

EKS Worker Node ki later Docker image kavali. Anduke image ni AWS ECR lo store chestam.

```
aws ecr create-repository \
  --repository-name myapp \
  --region ap-south-1
```

Ippudu AWS ECR lo:
myapp
ane repository ready.

---

11. ECR Login

Docker ni ECR tho authenticate chestam.

```
aws ecr get-login-password \
  --region ap-south-1 | \
docker login \
  --username AWS \
  --password-stdin \
  <ACCOUNT_ID>.dkr.ecr.ap-south-1.amazonaws.com
```

Login ayyaka Docker ECR ki image push cheyyagaladu.

---

12. Image Tag

Local image:
myapp:1.0
ni ECR repository address tho tag chestam.

```
docker tag myapp:1.0 \
<ACCOUNT_ID>.dkr.ecr.ap-south-1.amazonaws.com/myapp:1.0
```

---

13. ECR ki Image Push

```
docker push \
<ACCOUNT_ID>.dkr.ecr.ap-south-1.amazonaws.com/myapp:1.0
```

Ippudu image AWS ECR lo store ayyindi.
Linux Machine
↓

Docker Image
↓
docker push
↓
AWS ECR
↓
myapp:1.0
Ippudu infrastructure ready + Docker image ready.
_____--
PART 1 — EKS Infrastructure
3. EKS Cluster Create
Ippudu EKS cluster create chestam.

```
eksctl create cluster \
 --name my-cluster \
 --region ap-south-1 \
 --nodegroup-name my-nodes \
 --node-type t3.medium \
 --nodes 2
```

Ee command lo --nodes 2 icham kabatti example setup lo **2 worker nodes kosam Node Group** create avtundi. eksctl required EKS/networking configuration ni setup chestundi. Worker nodes EC2 instances ga run avtayi.
Ikkada important point: **Worker Nodes ikkade first create avtayi.** Ippudu Docker image, Pods, Deployment emi create cheyyaledu.
Linux Machine
↓
eksctl create cluster
↓
AWS
↓
EKS Cluster
↓
Networking
↓
Node Group
↓
EC2 Worker Nodes

---

4. EKS Cluster ki kubectl connect cheyyali
Cluster create ayyaka Linux machine lo kubectl ki mana EKS cluster address/configuration teliyali.

```
aws eks update-kubeconfig \
 --region ap-south-1 \
 --name my-cluster
```

Ippudu kubectl commands run chesthe avi mana EKS cluster **API Server** tho communicate chestayi.

---

5. Worker Nodes check cheyyali
Already create ayina worker nodes correct ga cluster lo join ayyayo check cheyyadaniki:

```
kubectl get nodes
```

Idi Nodes ni create cheyyadu. Existing worker nodes ni API Server daggara nunchi information teesukoni terminal lo chupistundi.
Example:

```
NAME     STATUS
node-1   Ready
node-2   Ready
```

Ready ante aa Worker Node Pods run cheyyadaniki available ga undi.
Ikkadi varaku infrastructure ready.
_____
PART 4 — Kubernetes + Helm
14. Helm Check
Helm available unda check:

```
helm version
```

Helm anedi Kubernetes applications ni package/deploy cheyyadaniki use chese tool.

---

15. Helm Chart Create

```
helm create myapp
```

Idi Linux machine lo Helm chart structure create chestundi.

```
myapp/
├── Chart.yaml
├── values.yaml
└── templates/
```

---

16. ECR Image Helm lo Configure
values.yaml lo ECR image specify chestam.

```
image:
  repository: <ACCOUNT_ID>.dkr.ecr.ap-south-1.amazonaws.com/myapp
```

tag: "1.0"
And replicas:
replicaCount: 3
Meaning:
Prathi Pod lo myapp:1.0 image use cheyyali, total 3 Pod replicas maintain cheyyali.

PART 5 — Helm → Kubernetes
17. Helm Install
Ippudu:
helm install myapp ./myapp
run chestam.
Ikkada Helm Kubernetes resources ni create cheyyadaniki EKS API Server tho communicate chestundi. Chart lo Deployment, Service etc. definitions unte avi Kubernetes cluster lo create avtayi.
Ikkadi nunchi Kubernetes actual application deployment start avtundi.

---

18. Deployment
Helm chart lo Deployment definition unte Kubernetes lo Deployment create avtundi.
Deployment lo:
replicas: 3
ante:
“Na application ki 3 Pods maintain cheyyali.”
Deployment desired state ni manage chestundi.
Check:
kubectl get deployments

---

19. ReplicaSet
Deployment controller required ReplicaSet ni create/manage chestundi.
Deployment
↓
ReplicaSet
ReplicaSet responsibility:
“Deployment cheppina number of Pods maintain cheyyali.”
Check:
kubectl get replicasets

---

20. Pods Create
ReplicaSet required Pods create chestundi.
3 replicas kabatti:
ReplicaSet
↓
Pod 1
Pod 2
Pod 3
Check:
kubectl get pods
Ikkada important: Pods ippudu Kubernetes objects ga create ayyayi. Kani Pod ye Worker Node meeda actual ga run avvali anedi Scheduler decide chestundi.

PART 6 — Pod → Worker Node
21. Scheduler
Scheduler unassigned Pods ni chusi suitable Worker Node select chestundi.
Mana Worker Nodes **already Step 3 lo create ayyayi.**
Example:
Existing Worker Nodes:

Node 1
Node 2

New Pods:

Pod 1
Pod 2
Pod 3
Scheduler decide cheyyachu:
Pod 1 → Node 1
Pod 2 → Node 2
Pod 3 → Node 1
Scheduler **new Worker Node create cheyyadu**. Existing Nodes lo suitable Node ni select chestundi.

---

22. Kubelet
Suppose:

Pod 1 → Node 1
Node 1 lo kubelet untundi.
Kubelet API Server nunchi Pod specification ni follow chesi container runtime ni use chesi Pod lo required containers run cheyyadaniki work chestundi.

---

23. Container Runtime
Node lo container runtime untundi. Example:
containerd
Kubelet runtime ni use chestundi.
Runtime ki Pod lo kavalsina image:
<ACCOUNT_ID>.dkr.ecr.ap-south-1.amazonaws.com/myapp:1.0
ani telustundi.
Image Worker Node lo already lekapothe ECR nunchi pull chestundi.
AWS ECR
↓
myapp:1.0
↓
Worker Node
↓
Container Runtime

---

24. Container Start
Image pull ayyaka container runtime aa image tho container create/start chestundi.
Worker Node
↓
Pod
↓
Container
↓
myapp:1.0
↓
Application Running
Ippudu actual application EKS Worker Node lo run avtundi.

PART 7 — Service + User
25. Service
Pods ki stable network access kosam Service create chestam.
Helm chart lo Service unte helm install time lone create ayyuntundi.
Check:
kubectl get svc
Service matching labels unna application Pods ni backend ga use chesi traffic ni correct Pod ki forward chestundi.

---

26. LoadBalancer
Application ni internet nunchi access cheyyali ante Service ni LoadBalancer type ga configure cheyyachu.
Example:
type: LoadBalancer
Check:
kubectl get svc
AWS environment lo corresponding load-balancing resource provision avvachu.

---

27. User Request
Ippudu user application URL access chestadu.
User
↓
DNS
↓
Load Balancer
↓
Service
↓
Pod
↓
Container
↓
Application
Application request process chesi response istundi.
Application
↓
Container
↓
Pod

↓
Service
↓
Load Balancer
↓
User

COMPLETE ORDER — EXACTLY
LINUX MACHINE
↓
aws configure
↓
EKS CLUSTER CREATE
↓
VPC / NETWORKING
↓
NODE GROUP
↓
EC2 WORKER NODES
↓
aws eks update-kubeconfig
↓
kubectl get nodes
↓
APPLICATION BUILD
↓
Dockerfile
↓
docker build
↓
DOCKER IMAGE
↓
ECR REPOSITORY
↓
docker tag
↓
docker push
↓
ECR IMAGE
↓
HELM CHART
↓
helm install
↓
DEPLOYMENT
↓
REPLICASET
↓
PODS
↓
SCHEDULER
↓
EXISTING WORKER NODE
↓
KUBELET
↓
CONTAINER RUNTIME
↓
ECR IMAGE PULL
↓
CONTAINER
↓
APPLICATION
↓
SERVICE
↓
LOAD BALANCER
↓
USER
Biggest point gurthupettuko

Worker Nodes Docker/ECR tarvatha create avvavu. EKS infrastructure setup time lo **Node Group dwara Worker Nodes create avtayi**.
Docker side separate:
Dockerfile
↓
docker build
↓
Docker Image
↓
ECR
Kubernetes deployment side:
Helm
↓
Deployment
↓
ReplicaSet
↓
Pod
↓
Scheduler
↓
Already-existing Worker Node
↓
Kubelet
↓
Container Runtime
↓
ECR Image
↓
Container
Kabatti **"Docker → Kubernetes → Worker Node create"** ani anukovaddu.
Correct meaning:
First EKS infrastructure lo Worker Nodes ready ga untayi. Tarvatha Docker image ECR lo ready chestam. Tarvatha Helm/Kubernetes Pod create chestayi. Scheduler aa Pod ni already unna Worker Node meeda place chestundi.

Avunu ra — application deployment context lo Docker image ready ayyaka Kubernetes use chestam. Kani **EKS cluster infrastructure ni mundhe create cheyyali**. Adi Docker tarvatha kaadu.
Nee practical story exact ga:
1. Linux Machine
↓
2. EKS Cluster + VPC + Node Group/Worker Nodes
↓
3. kubectl EKS ki connect
↓
4. Application Build
↓
5. Dockerfile
↓
6. docker build
↓
7. Docker Image
↓
8. ECR
↓
9. docker push
↓
10. Helm Chart
↓
11. helm install
↓
12. Kubernetes Deployment
↓
13. ReplicaSet
↓
14. Pods
↓
15. Scheduler
↓
16. Already-existing Worker Node

↓
17. Kubelet
↓
18. Container Runtime
↓
19. ECR nunchi Image Pull
↓
20. Container
↓
21. Application

So **“Kubernetes eppudu use chestam?”** ante:
Docker Image + ECR ready ayyaka application ni EKS lo deploy cheyyadaniki Kubernetes/Helm use chestam.
But **EKS cluster itself Docker tarvatha create cheyyadu**. EKS infrastructure first ready ga undali, appudu Docker image ni aa EKS cluster lo deploy chestam.
Simple ga:
Infrastructure first → Docker image next → Kubernetes deployment next.